Architecture SOA et Service Web

Chapitre5:

Composition de Service Web

Réalisé par: Dr. Ghallabi Sameh
l'Institut d'Informatique et Mathématique à Monastir
Cible : 3émé informatique

SOA

L'architecture orientée services (SOA) est de:

✓ Décomposition d'une application en un ensemble de fonctions basiques, appelées services.

✓ Décomposition d'un grand problème en parties plus petites.
✓Intégration dynamique des services
✓Décomposition en sous service avec possibilité de réutilisation.

⟶ SOA est une méthode de développement de logiciels qui utilise des composants logiciels appelés services pour créer des applications métier.

Problématique

Le besoin de l'utilisateur n'est pas satisfait par l'invocation d'un SW atomique, donc il faut combiner les fonctionnalités de plusieurs services.

⇇ Comment assembler ces petits services ensemble pour créer des services à valeur ajoutée ?

Composition de services

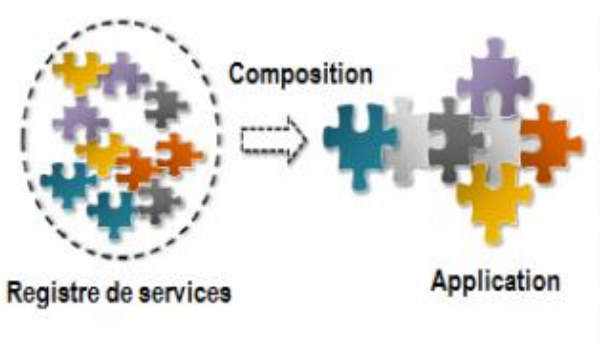
✓La composition consiste à combiner plusieurs services afin de générer un nouveau service à valeur ajoutée.

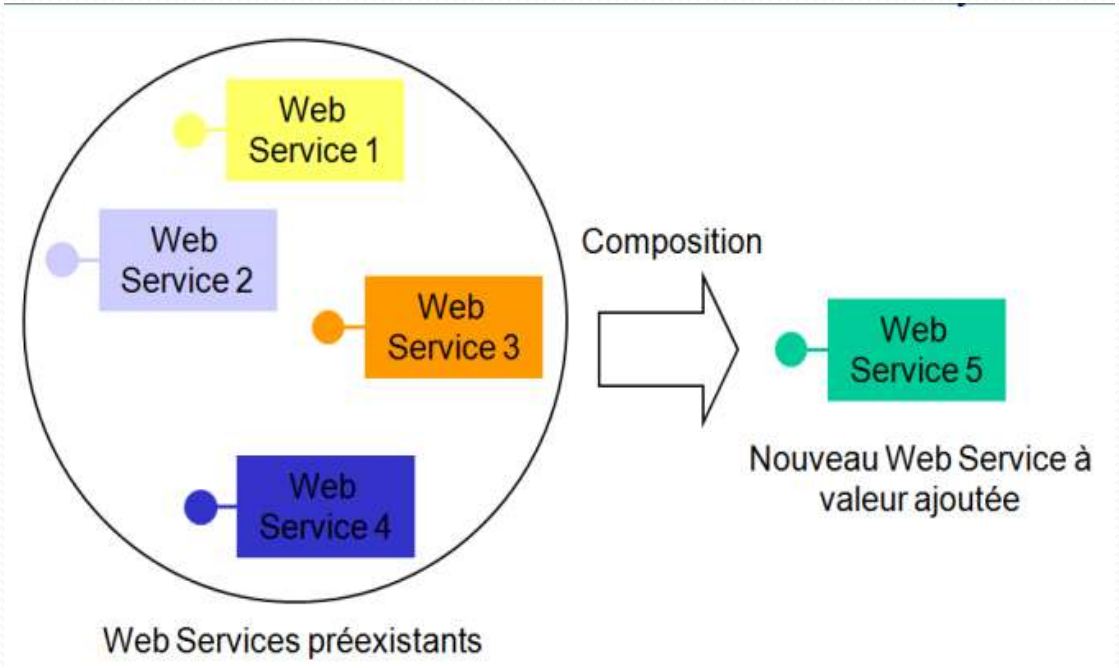
✓La composition de services est le mécanisme qui permet l'intégration des services pour construire un nouveau service web à valeur ajoutée.

✓Des services plus complexes peuvent être définis en composant d'autres services existants.

Composition de services

✓La composition de services est le mécanisme qui permet l'intégration des services pour construire un nouveau Web service à valeur ajoutée.





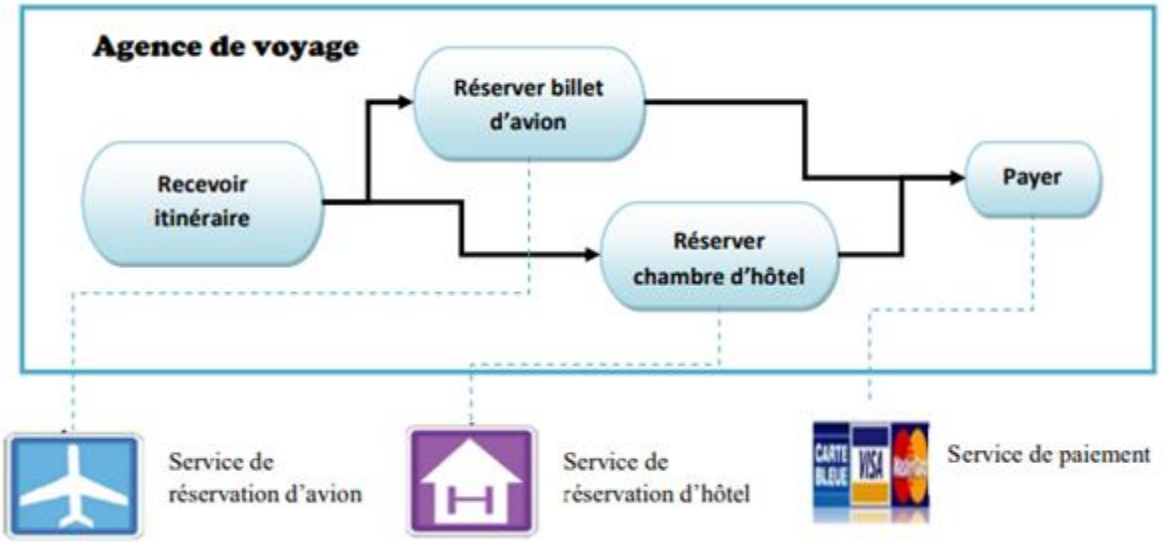
Composition de services

Implémentation d'une application (offerte comme service) dont la logique implique l'invocation d'opérations offertes par d'autres services.

-Le nouveau service est appelé service composite.

-Les services invoqués sont des composants de service.

Exemple:



Approches de composition

Deux approches principales pour la composition des services web :

- ✓Orchestration
- ✓ Chorégraphie

Orchestration de services

✓Les services peuvent être composés dans le but de réaliser un processus donné.

✓La composition de services par orchestration consiste à faire l'assemblage de services selon un ordre et un flux d'exécution.

✓Les orchestrations se présentent comme un formalisme pour composer des Services Web.

✓L'exécution d'une composition par orchestration est réalisée par un coordinateur de services.

✓Un coordinateur prend le control de tous les services web et coordonne l'exécution des différentes opérations des services web qui participent dans le processus.

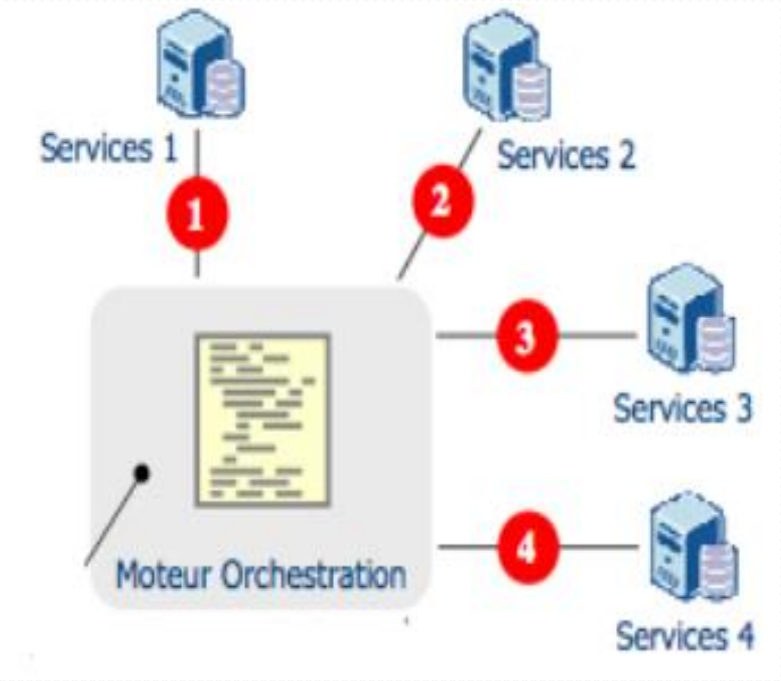
Orchestration de services

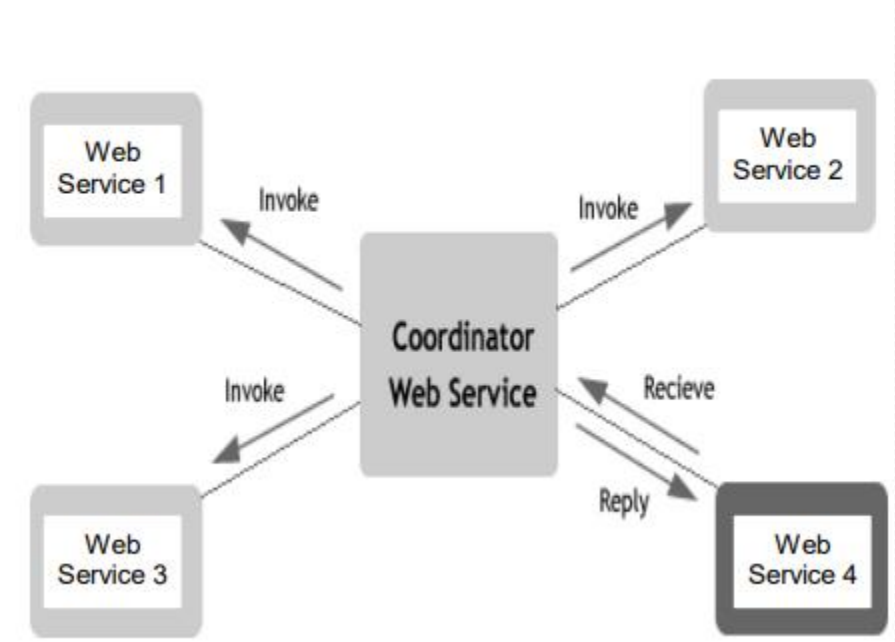
✓Pour réaliser un processus d'orchestration, on a besoin d'un langage et d'un moteur d'orchestration.

✓Ce type de composition permet de centraliser l'invocation des services web par un moteur d'exécution qui joue le rôle d'un chef d'orchestre.

✓Un moteur d'orchestration se charge d'appeler les services selon l'enchaînement désiré

Orchestration de services





Standards d'Orchestration

BPEL (Business Process Execution Language) ou BPEL4WS (BPEL for Web Services)

-Code qui exécute la séquence

-Exprimé en XML

Définit deux types d'activités:

-Activités de base : interagissant avec les services externes (invoke, receive, reply)

-Activités structurées : contrôle de flux du processus interne (flux séquentiel, condition, boucle…)

Orchestration de services

- Limites de l'Orchestration
  - Approche centralisée autour du moteur de composition
    - En cas de panne, plus de composition
  - Schéma de composition statique
    - En cas de changement dans les besoins, le schéma devient invalide

Chorégraphie de services

✓La Chorégraphie est un effort de collaboration dans lequel chaque participant du processus décrit l'itération qui l'appartient.

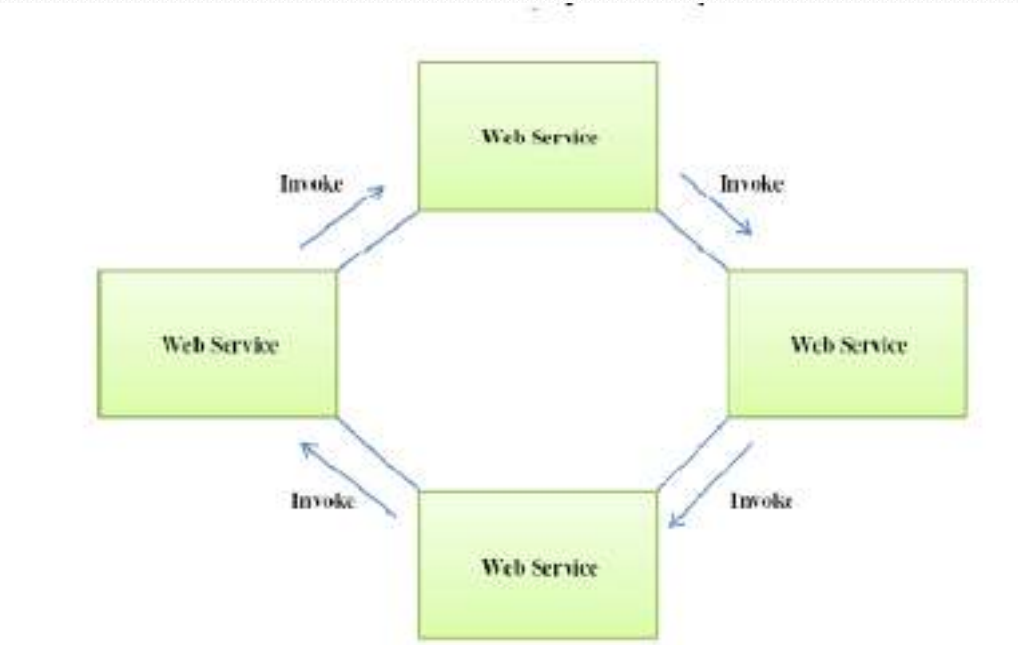
✓Le comportement global du processus est basé sur l'interaction des différentes parties, chacune suivant son propre rôle de manière autonome.

✓chaque service contribuant au service composite doit connaître sa logique (ex : attendre 50 s quand il est invoqué par le service X avant d'envoyer un message au service Y)

Chorégraphie de services

✓La Chorégraphie écrit le flux de messages échanges par un service web lors de son interaction avec d'autres services.

✓Il s'agit donc d'une manière décentralisée pour gérer une composition



Standards de la Chorégraphie

WS-CDL (Web Services Choregraphy Description Language)

✓Langage de description en XML

✓Décrit les messages qui sont impliqués dans l'échange collaboratif entre services

✓Ne définit pas de processus métier exécutable (contrairement à BPEL)

✓Un seul document WS-CDL spécifie la contribution d'un partenaire à l'échange de messages.

✓Contient un ensemble d'interactions, représentant les échanges de messages entre parties.

Chorégraphie de services

- Limites de la Chorégraphie
  - Collaborations et partenaires statiques
    - Si les besoins ou partenaires changent, les collaborations deviennent impossibles
  - Pas de langage pour exprimer les besoins
    - Les travaux existant proposent une chorégraphie statique

Chorégraphie / Orchestration

- Chorégraphie
  - Vision globale/distribuée
  - Collaboration publique entre participants
    - Description de l'ordre de messages échangés entre les participants
    - Définition de contraintes sur les interfaces des participants
  - Non exécutable tel quel (transformation en processus BPML)
  - WS-CDL (*WS-Choreography Description Language)*
- Orchestration
  - Vision centralisée et local à un processus
  - Implantation privée des participants
  - Exécutable
  - WS-BPEL (*WS-Business Process Execution Language)*

Langages de compositions

- Différents langages permettent de modéliser les processus métiers
- Fondés sur des concepts proches
  - Activités, opérations, appels de services
  - Echanges des messages, modification des messages
  - Gestion des erreurs, événements
- Syntaxe basée sur le format XML
- Outillés
  - Editeurs graphiques
  - Moteur d'exécution

BPEL : Business Process Execution language

Principe:

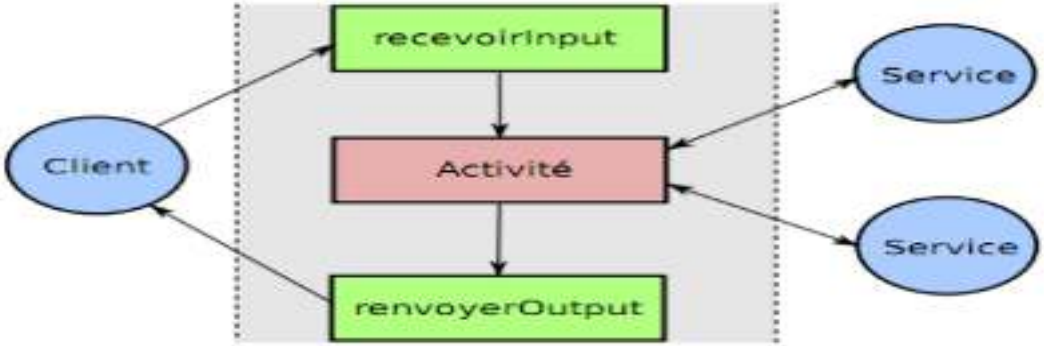
✓Le fichier BPEL définit le processus, ou l'enchaînement et la logique des actions.

✓Ce fichier est véritablement le code source de l'application que constitue le processus, le moteur d'orchestration agissant comme une machine virtuelle capable d'exécuter le code BPEL.

BPEL : Business Process Execution language

Processus BPEL:

- **Service Web, chef d'orchestre**, du point de vue client
- **Synchrone**
  - Le client se connecte au service (processus) et attend sa terminaison
- **Asynchrone**
  - Le client se connecte au service (processus) qui l'informe de sa terminaison (procédé callback)



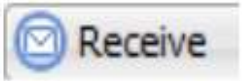
BPEL : Business Process Execution language

Eléments BPEL : Activités

- « **Invoke** » : pour appeler un service



- « **Receive** »: pour recevoir un message (initie le processus)



- « **Reply** » : pour renvoyer un message



- « **Throw** » : pour émettre une exception (message d'erreur)



BPEL : Business Process Execution language

Eléments BPEL : Activités

- « **Terminate** » : pour terminer le processus



- « **Wait** » : pour suspendre le processus pendant une durée déterminée



- « **Assign** » : pour manipuler des données
  - Ex : copier des valeurs entre variables ou utiliser des expressions XPath 1.0 pour effectuer des traitements (manipuler des strings…)



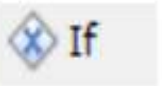
BPEL : Business Process Execution language

Eléments BPEL : Structures de contrôle

- « **Sequence** » : pour exécuter une activité séquentiellement après une autre



- « **Switch** » : structure de branchement selon des conditions booléennes écrites sous forme d'expressions XPath 1.0



- « **While** » : structure de boucle



BPEL : Business Process Execution language

Eléments BPEL : Structures de contrôle

- « **Pick** » : pour que le processus attende simultanément plusieurs types de messages avec un délai maximum d'attente, et effectue un traitement différent selon le cas



- « **Flow** » : pour paralléliser des activités (fork et join du diagramme d'activités UML)



BPEL : Business Process Execution language

Eléments BPEL :

✓ L'élément<process> est l'élément racine (au sens XML) du fichier BPEL.

✓L'élément <partnerLinks> permet de lier des actions définies dans le fichier WSDL (via partnerLinkType) au process BPEL

BPEL : Business Process Execution language

Eléments BPEL :

✓ L'élément<process> est l'élément racine (au sens XML) du fichier BPEL.

✓L'élément <partnerLinks> permet de lier des actions définies dans le fichier WSDL (via partnerLinkType) au process BPEL

BPEL : Business Process Execution language

Les partenaires d'un processus BPEL :

Deux types de partenaires d'un processus BPEL:

– Les services composants du processus BPEL à invoquer
– Les clients du service BPEL composite

✓Un élément <partnerLinks> exprime une relation entre le processus BPEL et les services composants à invoquer ou les clients du processus BPEL.

Attributs

-name : nom du partnerLink
-myRole : rôle du processus
-partnerRole : rôle du partenaire
- partnerLinkType : type du partnerLink défini dans la description WSDL.

BPEL : Business Process Execution language

Les rôles dans <partnerLinks> :

Les attributs myRole et partnerRole peuvent être définis en même temps.

✓Quand seulement myRole est spécifié → lien pour invoquer le service BPEL composite

✓Quand seulement partnerRole est spécifié → lien pour invoquer un service composant

BPEL : Business Process Execution language

Eléments BPEL : Variables

✓BPEL définit la notion de variables qui permet de manipuler les messages des interactions (envoyés et reçus) entre les partenaires.

✓Une variable est définie par les attributs suivant:

- name : nom de la variable
- type: (ex : type schéma XML) ou messageType déclarés dans un fichier WSDL

BPEL : Business Process Execution language

Activité <Invoke >

- ✓L'activité invoque une opération (request/response) d'un partenaire .
- ✓Utilise un message en entrée et éventuellement un message en sortie.

```
<invoke
        name="…"                    → nom de l'activité
        partnerLink="…"             → lien identifiant  le partenaire
        portType="…."               → portType de l'operation
        operation="…."              → l'opération à invoquer
        inputvariable="…."          → variable contenant le message en entrée
        outputvariable="…." />      → variable contenant le message en sortie
```

BPEL : Business Process Execution language

Activité <Receive>

L'activité <Receive> est une activité bloquante en attente d'un message à envoyer par un partenaire/client

```
<receive
        name="…"
        createInstance="yes"
        partnerLink="…"
        portType="…."
        operation="…."
        variable="…." />
```

- ➔ Nom de l'activité
- ➔ Dans ce cas, une nouvelle instance sera créée
- ➔ Lien identifiant le partenaire
- ➔ l'operation que le processus doit implementer
- ➔ variable où stocker le message recu

BPEL : Business Process Execution language

Activité <reply >

L'activité <reply > permet d'envoyer (répondre) un message à un partenaire

```
<reply
        name="…"
        partnerLink="…"
        portType="…."
        operation="…."
        variable="…." />
```

→ Nom de l'activité

→ Lien identifiant le partenaire

→ l'opération que le processus doit implémenter

→ variable contenant le message à envoyer

BPEL : Business Process Execution language

Activité <assign>

L'activité <assign> permet d'affecter la valeur d'une variable à partir de la valeur d'une autre variable.

```
<assign>
   <copy>
      <from variable="...."/>
      <to variable="...."/>
   </ copy >
</ assign >
```

➔ l'activité <assign> peut contenir plusieurs éléments <copy>

➔ Possibilité d'utiliser des expressions Xpath et transformations XSL pour extraire/construire les messages appropriés

BPEL : Business Process Execution language

Exemple:

```
<?xml version='1.0' encoding='UTF-8'?>

<process xmlns="http://schemas.xmlsoap.org/ws/2003/03/business-process/">
        <partnerLinks>
                <partnerLink myRole="dfltMyRole" name="dfltName" partnerLinkType="dfltPartnerLinkType"
partnerRole="dfltPartnerRole"></partnerLink></partnerLinks>
        <partners>
                <partner name="dfltName"></partner>
                <partnerLink name="dfltName"></partnerLink></partners>
        <variables>
                <variable messageType="dfltMessageType" name="dfltName"></variable></variables>
        <sequence>
                <receive createInstance="yes" name="Commande du client" operation="dfltOperation"
partner="" partnerLink="dfltPartnerLink" portType="dfltPortType" variable=""></receive>
                <empty name="Enregistrement de la commande sur la caisse"></empty>
                <empty name="Afficher commande en cuisine (pour préparation)"></empty>
                <empty name="encaissement"></empty>
                <switch name="mode de paiement">
                        <otherwise>
                                <empty name="montant exact"></empty>
                        </otherwise>
                        <case condition="carte">
                                <empty name="Contrôler carte"></empty>
                                <switch name="code PIN?">
                                        <otherwise>
                                                <empty name="Valider"></empty>
                                        </otherwise>
                                        <case condition="faux">
                                                <empty name="Refaire code"></empty>
                                                <switch name="correct?">
                                                        <otherwise>
                                                                <empty name="Valider"></empty>
                                                        </otherwise>
                                                </switch>
                                        </case>
                                </switch>
                                <empty name="Ejecter carte"></empty>
                        </case>
                </switch>
                <empty name="Imprimer quittance"></empty>
                <empty name="Transfert données ordinateur central"></empty>
                <invoke operation="dfltOperation" partner="" partnerLink="dfltPartnerLink"
portType="dfltPortType"></invoke>
        </sequence>
</process>
```

BPEL : Business Process Execution language

Pour faire du BPEL : Outils

- Deux catégories d'outils sont à distinguer
  - Editeur graphique de processus BPEL
  - Moteur BPEL intégré dans la majorité des serveurs d'application

- A noter que les solutions BPEL proposées par les éditeurs de logiciels fournissent à la fois l'éditeur et le serveur

- Les éditeurs graphiques sont généralement intégrables dans les environnements de développement (Eclipse, Netbeans, Visual Studio, …)

- Dans ce cours, nous utiliserons Eclipse BPEL designer comme éditeur graphique et Apache ODE comme moteur d'exécution